Research Agent — Improvement and Engineering Roadmap

Priorities for reliability, RAG quality, testing, security, evaluation, and portfolio readiness

Prepared: 10 October 2026

Repository: https://github.com/BGRAMPAGE/research_agent

1. Objective and target architecture

- Improve the existing repository, which has two profiles: Deep Research Agent (web research and cited report generation) and Paper Library (RAG over a curated AI/ML paper corpus). Treat each item as a validation task against the exact current commit, not as an assertion that every issue is present.
- Target flow: Chainlit UI → profile router → (A) LangGraph research workflow or (B) Paper Library RAG service. Shared components should include configuration, provider factory, database access, logging, caching, metrics, and evaluation.
- Research flow: plan → web search → normalize/score sources → synthesize evidence → generate cited report. Paper Library flow: validate question → embed query → retrieve pgvector chunks → build context → generate answer → validate citations → return answer with source references.

2. P0 — Reproducible setup

- Fix README clone URL to https://github.com/BGRAMPAGE/research_agent and ensure the documented folder tree matches the real repository.
- Document supported Python version, Windows PowerShell and Linux/macOS setup, dependency install, database setup, vector extension, ingestion procedure, launch command, and troubleshooting.
- Add .env.example with variable names and safe placeholders. Keep .env, keys, passwords, tokens, private documents, and database dumps out of Git.
- Document optional dependencies: Ollama/local models, hosted model providers, web-search APIs, PostgreSQL/pgvector, and report export tools.
- Add a fresh-clone smoke test: imports work, configuration is validated, database connectivity is checked, and both profile routes are confirmed.

3. P1 — Workflow correctness and resilience

- Write routing tests proving Deep Research invokes the research graph and Paper Library invokes answer_from_papers. Define behavior for missing or unknown profile names.
- Verify the checkpointer used by the live research path. If runs must resume after restart, wire persistent SQLite/PostgreSQL checkpointing into the graph actually created by the UI; test interruption and resume.
- Distinguish empty retrieval from database connection failure, SQL/schema errors, embedding failures, and provider errors. Use typed outcomes or dedicated exceptions.
- Add explicit timeouts and bounded retries with backoff for transient model/search errors. Avoid infinite retries and retries for invalid credentials or malformed requests.
- Keep blocking database/embedding/model work off the async event loop where appropriate. Test connection pool behavior and a small number of concurrent sessions.
- Validate structured model outputs, limit report size, sanitize export filenames, and clean temporary files.

4. P1 — RAG quality and evidence integrity

- Create a versioned evaluation set of 30–50 questions over the indexed papers. Record expected supporting paper/chunk and questions that should be answered with insufficient evidence.
- Calibrate an abstention policy on the evaluation set. If retrieval is empty or weak, say evidence is insufficient; do not rely on an arbitrary universal similarity threshold.
- Preserve stable paper IDs, title, chunk ID, URL/DOI, and retrieval score through context construction and generation.
- Validate every generated citation against the retrieved source list. Flag unknown IDs and unsupported/malformed citations; where practical, link claims to evidence passages.
- Benchmark the current vector baseline against chunk size/overlap variants, embedding models, hybrid lexical + vector search, and optional cross-encoder reranking. Re-embed the entire corpus when changing embedding model/dimension.
- Treat retrieved paper text as untrusted data, not instructions. Protect against prompt injection in retrieved documents.

5. P1 — Automated tests and CI

- Unit tests: configuration, profile router, context builder, citation validator, source scoring, report formatting, embedding dimensions, and error mapping.
- Integration tests: one Paper Library query against a test database or mocked repository; one research graph run with mocked search/model tools.
- Failure tests: empty corpus, database offline, malformed model output, timeout, no web results, invalid API key, unsupported citation, and oversized upload.
- Add pytest and GitHub Actions to run deterministic tests on push/pull request. Keep paid/network-dependent tests in an optional job. Consider Ruff and type checking incrementally.

6. P2 — Security, observability, and evaluation

- Use structured logs with run/request ID, profile, stage, duration, provider, retrieval count, and status. Do not log API keys, private document contents, or full sensitive prompts by default.
- Show concise user-facing errors; retain tracebacks in server logs and show a correlation ID instead.
- Track latency, token use, provider errors, web-search count, cache hit rate, retrieval scores, and estimated cost when available.
- Limit upload size, file types, query length, concurrency, and report size. Validate file content instead of trusting extensions.
- For public deployment, add authentication, authorization, rate limits, secret management, and safe CORS. Never expose PostgreSQL directly to the public internet.
- Retrieval metrics: Recall@5 and MRR@10. Answer metrics: faithfulness, relevance, completeness, and citation validity/support. Operations metrics: median/p95 latency, error rate, and cost per successful query where measurable.
- Set acceptance thresholds after collecting a baseline. Publish the dataset size, method, and limitations with the results.

7. Suggested layout and delivery sequence

- Keep existing structure where possible. A target layout could include app.py (UI), src/graph.py (research graph), src/rag_chat.py (RAG service), src/retrieval/ (retriever/context builder), src/providers/, src/config.py, src/observability.py, tests/, scripts/ (ingestion/evaluation), docs/, .env.example, Dockerfile, and README.
- Milestone 1: README, .env.example, setup smoke test. Milestone 2: profile routing tests, typed retrieval errors, safe errors, checkpoint wiring. Milestone 3: evaluation set, citation validation, abstention policy, baseline metrics. Milestone 4: compare hybrid retrieval/reranking/chunking. Milestone 5: CI, Docker smoke test, demo screenshots and architecture diagram.

8. Real-world fit and definition of done

- Good fit after improvements: personal research assistant, student research tool, internal knowledge-base prototype, or small-team proof of concept using public/non-sensitive papers.
- Not ready for confidential/high-impact use without authentication, tenant isolation, retention/deletion rules, backups, rate limits, monitoring, incident response, and defined service-level expectations.
- Done checklist: fresh clone runs from README; both profiles route correctly; checkpoint behavior matches documentation; retrieval errors are distinct from empty results; citations are validated; evaluation data/results exist; tests/CI pass; no secrets are committed; demo and limitations are documented.

Maintenance note

Before implementation, verify the exact repository commit and re-check provider pricing, free-tier quotas, terms, and security settings. Free-tier limits change.